

Project Guideline

Solar Irradiance Prediction (SIP) Using Large Language Models

Revised project format: LLM-based numerical prediction with conventional ML baselines

Project Purpose

Students will design, implement, and evaluate an LLM-based solar irradiance forecasting pipeline. The LLM must be part of the prediction mechanism, not only a writing assistant or code helper. The project should demonstrate whether an LLM-based approach can produce useful numerical forecasts of solar irradiance and how it compares with conventional machine learning baselines.

Teaming

- Each team must consist of 2–3 students.
- Please email your team composition and preferred topic direction by **Tuesday, June 17.**
- Each team should identify one member responsible for data/preprocessing, one for modeling/evaluation, and one for presentation/report integration. Roles may overlap for 2-person teams.

Data Source(s)

- Use public datasets such as NASA POWER, NREL NSRDB, PVGIS, NOAA/local weather stations, or a university/local weather station dataset.
- The dataset should include historical solar irradiance and relevant meteorological variables, such as temperature, humidity, cloud cover, wind speed, precipitation, pressure, and solar zenith angle if available.
- The recommended prediction target is hourly Global Horizontal Irradiance (GHI). Teams may also predict Direct Normal Irradiance (DNI), Diffuse Horizontal Irradiance (DHI), or PV power output if the data support it.
- Use at least one full year of data when possible. If using a smaller dataset, clearly justify the limitation.

Problem Statement and Modeling Objectives

- Define the forecasting target clearly, for example: predict hourly GHI for the next 1 hour, 6 hours, or 24 hours.
- State whether the task is forecasting, regression, or classification. The preferred task is numerical time-series forecasting.
- Define the input window, forecast horizon, location, temporal resolution, and meteorological variables used by the model.
- State the practical value of the forecast, such as PV scheduling, battery dispatch, microgrid operation, or solar-powered IoT scheduling.

Required LLM-Based Prediction Component

- The LLM must generate or directly contribute to numerical irradiance predictions on the validation/test set.
- Acceptable LLM approaches include:
 - Prompt-based forecasting: convert recent time-series observations and weather variables into a structured prompt and ask the LLM to output future GHI values.
 - Few-shot forecasting: include several historical input-output examples in the prompt before requesting a new forecast.

- Retrieval-augmented forecasting: retrieve similar historical weather/irradiance days and provide them to the LLM as context.
- LLM-assisted feature or rule generation: use the LLM to construct interpretable forecasting features, then test them in a predictive model.
- Fine-tuned or adapted LLM/foundation model: use an LLM-style transformer model adapted for time-series prediction, if resources allow.
- Not acceptable as the only LLM component: using ChatGPT only to write code, explain results, create slides, or edit the report.
- The LLM output must be machine-readable. For example, require JSON or CSV-like numeric output with timestamps and predicted GHI values.
- Prevent data leakage. Do not include future measured irradiance values in the prompt, retrieved context, training features, or validation labels.

Suggested LLM Prompt Structure

Teams should include their final prompt template in the report. A typical structure is:

Task: Predict hourly GHI for the next H hours.

Location and season: [site, month, day type].

Recent observations: [timestamp, GHI, temperature, humidity, cloud cover, wind speed].

Forecast weather inputs if available: [future temperature/cloud cover/etc.].

Rules: Return only valid JSON with timestamp and predicted_GHI_Wm2. Do not explain. Do not use values outside 0–1200 W/m².

Graphs and Preprocessing

- Create time-series plots of solar irradiance at daily, weekly, and seasonal/yearly resolution.
- Provide summary statistics and visualizations: histograms, correlation heatmaps, missing-data patterns, and clear-sky versus cloudy-day examples.
- Document preprocessing steps: time alignment, missing-value handling, normalization/scaling, outlier removal, and feature engineering.
- For LLM forecasting, document how numerical time-series data are converted into text or structured tokens, including prompt window length and token-budget constraints.
- Include temporal features such as hour of day, day of year, season, weekend/weekday if relevant, and solar geometry features if available.

Baseline Models for Comparison

- Each team must compare the LLM-based method with at least two non-LLM baselines.
- Recommended baselines: persistence model, climatology/seasonal average, Linear Regression, Random Forest, XGBoost, SVR, LSTM, or a standard neural network.
- Justify the selected baselines and explain their assumptions, training methods, and validation approach.
- The final conclusion should state whether the LLM improves prediction accuracy, interpretability, usability, or workflow efficiency compared with the baselines.

Training, Validation, and Evaluation

- Use chronological train/validation/test splitting. Avoid random shuffling across time because it can leak future information into training.
- Recommended split: 70% training, 15% validation, 15% test, or a walk-forward validation design.
- Report metrics such as RMSE, MAE, normalized RMSE, R^2 , and optionally MAPE or sMAPE when appropriate.
- Report metrics by forecast horizon, for example 1-hour, 6-hour, and 24-hour ahead.
- Perform error analysis for clear-sky, partly cloudy, and cloudy/rainy conditions when labels are available.
- For LLM methods, report additional practical metrics such as prompt length, inference cost if applicable, latency, output-parsing failure rate, and reproducibility across repeated runs.

Ablation and Sensitivity Study

- Compare zero-shot, few-shot, and retrieval-augmented prompts if feasible.
- Test different input-window lengths, such as previous 6, 12, 24, or 48 hours.
- Evaluate whether adding meteorological variables improves the LLM forecast compared with using GHI history alone.
- Discuss cases where the LLM fails, such as sudden cloud transients, storm days, missing weather data, or extreme seasonal changes.

Application Context

- Propose a real or hypothetical application such as solar forecasting for grid-tied PV systems, battery energy management, microgrid dispatch, demand response, or smart scheduling for solar-powered IoT devices.
- Explain how accurate predictions improve system performance, economics, resilience, or operational planning.
- Discuss how prediction errors could affect decisions, for example overcharging/undercharging a battery or incorrectly scheduling flexible loads.

Ethical and Practical Considerations

- Discuss data bias, geographic representativeness, and limitations of climate/weather datasets.
- Identify risks of overfitting, prompt leakage, hallucinated outputs, and overconfidence in high-stakes applications such as grid planning or operational dispatch.
- If using a commercial LLM API, discuss data privacy, reproducibility, cost, model-version changes, and rate-limit constraints.
- State how the team validated that the LLM predictions are physically reasonable, such as nonnegative irradiance, nighttime near-zero irradiance, and realistic upper bounds.

Final Report and Presentation

- Written report should include all sections above, references, prompt templates, model settings, evaluation results, plots, and an appendix with code summary or repository link.
- Include a table comparing LLM and baseline model performance on the same test set.
- Include at least one example showing the LLM input prompt and the parsed numerical forecast output.
- Team presentation will be scheduled in the final week of the summer program.
- Presentation should include: problem motivation, dataset, preprocessing, LLM design, baselines, results, error analysis, application impact, and lessons learned.

Recommended Minimum Workflow

1. Select dataset, site, target variable, temporal resolution, and forecast horizon.
2. Clean and visualize the data; create train/validation/test splits in chronological order.
3. Implement at least two baseline models.
4. Design the LLM prompt or LLM-based pipeline for numerical prediction.
5. Parse and validate LLM outputs automatically.
6. Evaluate LLM and baselines using the same test set and metrics.
7. Perform error analysis and discuss engineering implications.

SIP Project Evaluation

Team: _____	Date: _____
Project Title: _____	Evaluator: _____

Rating scale: U = unacceptable, P = poor, A = acceptable, G = good, E = excellent

Category	U	P	A	G	E
1. Were the project scope, target variable, forecast horizon, and goals clearly presented?					
2. Was the analysis based on appropriate mathematical, scientific, and engineering principles?					
3. Was the dataset selection, preprocessing, visualization, and missing-data handling appropriate?					
4. Was the LLM-based prediction method clearly designed, justified, and implemented as an actual forecasting component?					
5. Were non-LLM baseline models considered and fairly compared with the LLM method?					
6. Were realistic constraints considered, such as data leakage, token budget, API cost, latency, reproducibility, and physical bounds?					
7. Were appropriate performance metrics reported, including RMSE, MAE, R^2 , and horizon-specific results?					
8. Was the validation strategy sound, using chronological splits or walk-forward testing without future-data leakage?					
9. Were results interpreted clearly, including error analysis and application implications?					
10. Were grammar, spelling, figures, tables, and presentation quality at a professional level?					
11. Overall assessment of project quality.					

Comments